

ADA Lab Programs

Programs + Output

USN: 1WN24CS232

Sl. no.:	Title
1	Quicksort
2	Heapsort
3	0/1 Knapsack - DP
4	Floyd's algo
5	Johnson Trotter
6	Prim's algo
7	Kruskal's algo
8	Knapsack – Greedy
9	Dijkstra's algo
10	N Queen's algo
11	Leet Code Exercises

Sort a given set of N integer elements using **Quick Sort** technique and compute its time taken.

The image shows two screenshots of the Code::Blocks IDE. The top screenshot displays the source code for a Quick Sort implementation in C++. The bottom screenshot shows the same code with a terminal window open, displaying the program's output.

Source Code (Quick sort.c):

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4 int partition(int a[], int low, int high) {
5     int pivot=a[low],i=low+1,j=high,temp;
6     while(1) {
7         while(i<=high && a[i]<=pivot) i++;
8         while(a[j]>pivot) j--;
9         if(i<j) {
10             temp=a[i];
11             a[i]=a[j];
12             a[j]=temp;
13         } else
14             break;
15     }
16     temp=a[low];
17     a[low]=a[j];
18     a[j]=temp;
19     return j;
20 }
21 void quicksort(int a[], int low, int high) {
22     if(low<high) {
23         int p=partition(a,low,high);
24         quicksort(a,low,p-1);
25         quicksort(a,p+1,high);
26     }
27 }
28 int main() {
29     int n,i;
30     clock_t start,end;
31     double time_taken;
32     printf("Enter number of elements: ");
33     scanf("%d",&n);
```

Execution Output:

```
Enter number of elements: 10
Sorted elements:
1230 2548 3866 4829 4859 5287 6319 7269 7349 8500
Time taken = 0.000000 seconds
Process returned 0 (0x0)   execution time : 2.936 s
Press any key to continue.
```

Sort a given set of N integer elements using **Heap Sort** technique and compute its time taken.

The screenshot displays the Code::Blocks IDE with a C++ project named 'Heap sort.c'. The code implements the Heap Sort algorithm, which includes a `heapify` function to maintain the heap property and a `heapsort` function to sort the array. The `main` function takes the number of elements as input, generates random integers, and measures the execution time using `clock_t`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4 void heapify(int a[], int n, int i) {
5     int largest=i, l=2*i+1, r=2*i+2, temp;
6     if(l<n && a[l]>a[largest]) largest=l;
7     if(r<n && a[r]>a[largest]) largest=r;
8     if(largest!=i) {
9         temp=a[i];
10        a[i]=a[largest];
11        a[largest]=temp;
12        heapify(a,n,largest);
13    }
14 }
15 void heapsort(int a[], int n) {
16     int i,temp;
17     for(i=n/2-1;i>=0;i--)
18         heapify(a,n,i);
19     for(i=n-1;i>0;i--) {
20         temp=a[0];
21         a[0]=a[i];
22         a[i]=temp;
23         heapify(a,i,0);
24     }
25 }
26 int main() {
27     int n,i;
28     clock_t start,end;
29     double time taken;
30     printf("Enter number of elements: ");
31     scanf("%d",&n);
32     int a[n];
33     srand(time(0));
```

The execution output shows the program running successfully with 10 elements. The sorted elements are: 105 989 1002 3222 3549 4472 4654 7329 8707 9078. The time taken is 0.000000 seconds. The process returned 0 (0x0) and the execution time was 3.984 s.

```
Enter number of elements: 10
Sorted elements:
105 989 1002 3222 3549 4472 4654 7329 8707 9078
Time taken = 0.000000 seconds
Process returned 0 (0x0)   execution time : 3.984 s
Press any key to continue.
```

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>

int main() {
    int n, w, i, j;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int wt[n+1], val[n+1];
    printf("Enter weights:\n");
    for(i=1; i<=n; i++)
        scanf("%d", &wt[i]);
    printf("Enter profits:\n");
    for(i=1; i<=n; i++)
        scanf("%d", &val[i]);
    printf("Enter capacity: ");
    scanf("%d", &w);
    int dp[n+1][w+1];
    for(i=0; i<=n; i++) {
        for(j=0; j<=w; j++) {
            if(i==0 || j==0)
                dp[i][j]=0;
            else if(wt[i]<=j) {
                int include=val[i]+dp[i-1][j-wt[i]];
                int exclude=dp[i-1][j];
                dp[i][j]=(include>exclude)?include:exclude;
            } else
                dp[i][j]=dp[i-1][j];
        }
    }
    printf("\nDP Table:\n");
    for(i=0; i<=n; i++) {
        for(j=0; j<=w; j++)
            printf("%4d", dp[i][j]);
        printf("\n");
    }
}
```

Build file: "no target" in "no project" (compiler: unknown) ===
Build finished: 0 error(s), 0 warning(s) (0 minute(s), 1 second(s)) ===

D:\Sem 4\ADA\LAB\01 Knapsack C++ Windows (CR+LF) WINDOWS-1252 Line 36, Col 2, Pos 985 Insert Modified Read/Write default

```
#include <stdio.h>

int main() {
    int n, w, i, j;
    printf("Enter number of items: 4\n");
    scanf("%d", &n);
    printf("Enter weights: 2 3 4 5\n");
    int wt[n+1], val[n+1];
    for(i=1; i<=n; i++)
        scanf("%d", &wt[i]);
    printf("Enter profits: 4 5 6\n");
    for(i=1; i<=n; i++)
        scanf("%d", &val[i]);
    printf("Enter capacity: 7\n");
    scanf("%d", &w);
    printf("Enter DP Table:\n");
    for(i=0; i<=n; i++) {
        for(j=0; j<=w; j++)
            printf("%4d", dp[i][j]);
        printf("\n");
    }
    printf("Maximum Profit = 9\n");
    return 0;
}
```

Build file: "no target" in "no project" (compiler: unknown) ===
Build finished: 0 error(s), 0 warning(s) (0 minute(s), 1 second(s)) ===

D:\Sem 4\ADA\LAB\01 Knapsack C++ Windows (CR+LF) WINDOWS-1252 Line 36, Col 2, Pos 985 Insert Read/Write default

Implement All Pair Shortest paths problem using **Floyd's** algorithm.

```
#include <stdio.h>
#define INF 999
int main() {
    int n, i, j, k;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int a[n][n];
    printf("Enter cost matrix:\n");
    for(i=0; i<n; i++)
        for(j=0; j<n; j++)
            scanf("%d", &a[i][j]);
    for(k=0; k<n; k++)
        for(i=0; i<n; i++)
            for(j=0; j<n; j++)
                if(a[i][k]+a[k][j]<a[i][j])
                    a[i][j]=a[i][k]+a[k][j];
    printf("\nAll Pair Shortest Path Matrix:\n");
    for(i=0; i<n; i++)
        for(j=0; j<n; j++)
            printf("%d", a[i][j]);
    printf("\n");
    return 0;
}
```

Build file: "no target" in "no project" (compiler: unknown) ===
Build finished: 0 error(s), 0 warning(s) (0 minute(s), 1 second(s)) ===

CODE

D:\Sem 4\ADA\LAB\Floyd.c C/C++ VERIFIED (CR+LF) WINDOWS-1252 Line 24, Col 2, Pos 627 Insert Modified Read/Write default

```
#include <stdio.h>
#define INF 999
int main() {
    int n, i, j, k;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int a[n][n];
    printf("Enter cost matrix:\n");
    for(i=0; i<n; i++)
        for(j=0; j<n; j++)
            scanf("%d", &a[i][j]);
    for(k=0; k<n; k++)
        for(i=0; i<n; i++)
            for(j=0; j<n; j++)
                if(a[i][k]+a[k][j]<a[i][j])
                    a[i][j]=a[i][k]+a[k][j];
    printf("\nAll Pair Shortest Path Matrix:\n");
    for(i=0; i<n; i++)
        for(j=0; j<n; j++)
            printf("%d", a[i][j]);
    printf("\n");
    return 0;
}
```

D:\Sem 4\ADA\LAB\Floyd.exe

Enter number of vertices: 4
Enter cost matrix:
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0

All Pair Shortest Path Matrix:
0 5 8 0
999 0 3 4
999 999 0 1
999 999 999 0

Process returned 0 (0x0) execution time : 418.096 s
Press any key to continue.

Build file: "no target" in "no project" (compiler: unknown) ===
Build finished: 0 error(s), 0 warning(s) (0 minute(s), 0 second(s)) ===

D:\Sem 4\ADA\LAB\Floyd.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 24, Col 2, Pos 627 Insert Read/Write default

Implement **Johnson Trotter** algorithm to generate permutations.

The image shows a code editor window titled "Johnson Trotter.c - Code::Blocks 25.03". The code implements the Johnson-Trotter algorithm to generate permutations of a set of numbers. The algorithm uses a recursive approach with a "mobile" element and a "direction" array to generate all possible permutations.

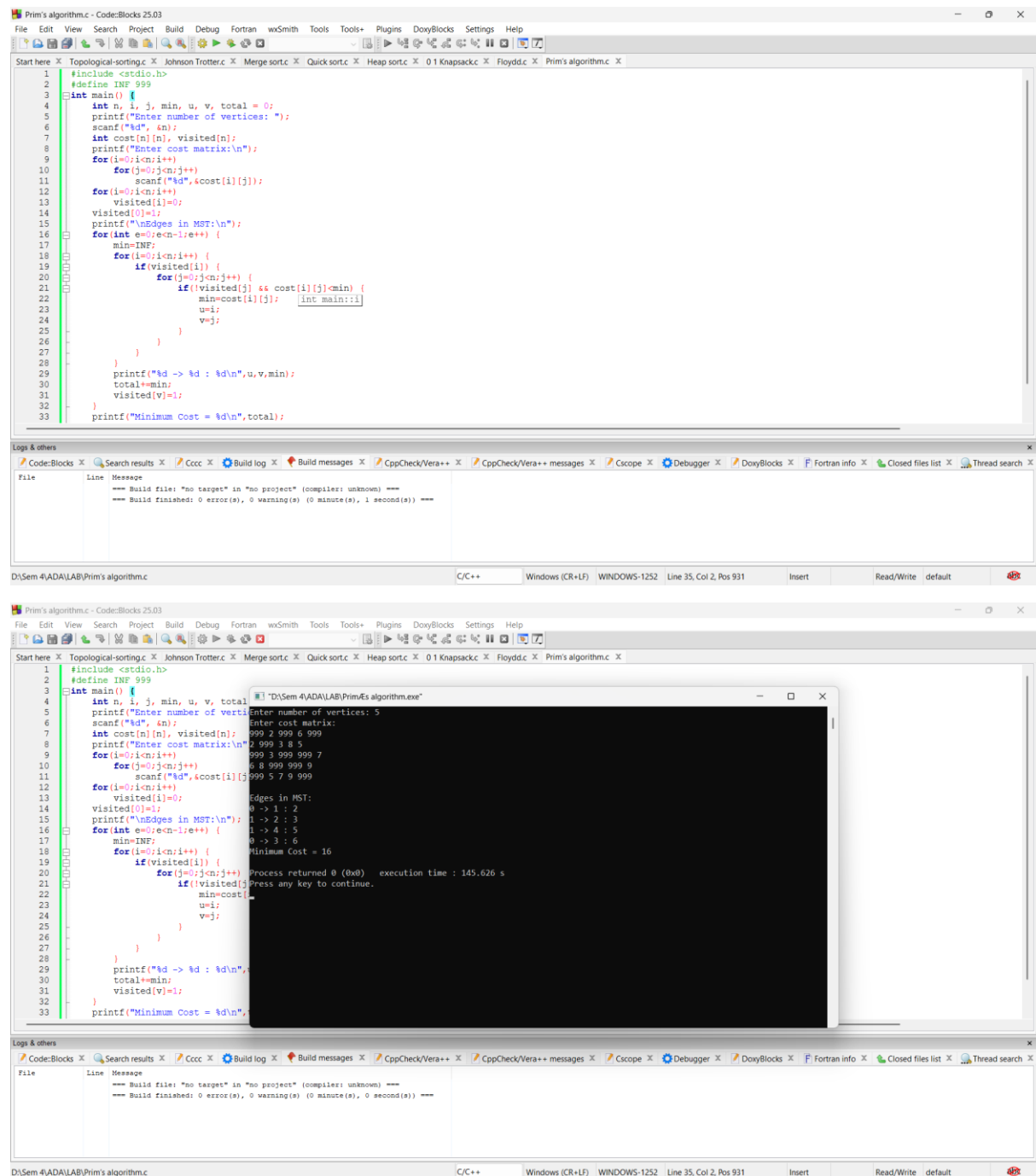
```
1 #include <stdio.h>
2 int main() {
3     int n, i, j, mobile, mobile_pos, temp;
4     printf("Enter n: ");
5     scanf("%d", &n);
6     int a[n], dir[n];
7     for(i = 0; i < n; i++) {
8         a[i] = i + 1;
9         dir[i] = -1;
10    }
11    while(1) {
12        for(i = 0; i < n; i++)
13            printf("%d ", a[i]);
14        printf("\n");
15        mobile = 0;
16        mobile_pos = -1;
17        for(i = 0; i < n; i++) {
18            if(dir[i] == -1 && i > 0 && a[i] > a[i - 1] && a[i] > mobile) {
19                mobile = a[i];
20                mobile_pos = i;
21            }
22            if(dir[i] == 1 && i < n - 1 && a[i] > a[i + 1] && a[i] > mobile) {
23                mobile = a[i];
24                mobile_pos = i;
25            }
26        }
27        if(mobile == 0)
28            break;
29        if(dir[mobile_pos] == -1) {
30            temp = a[mobile_pos];
31            a[mobile_pos] = a[mobile_pos - 1];
32            a[mobile_pos - 1] = temp;
33            temp = dir[mobile_pos];
34            dir[mobile_pos] = -temp;
35        } else {
36            temp = a[mobile_pos];
37            a[mobile_pos] = a[mobile_pos + 1];
38            a[mobile_pos + 1] = temp;
39            temp = dir[mobile_pos];
40            dir[mobile_pos] = -temp;
41        }
42    }
43 }
```

The execution output shows the following permutations for n=3:

```
Enter n: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3
```

Process returned 0 (0x0) execution time : 11.928 s
Press any key to continue.

Find Minimum Cost Spanning Tree of a given undirected graph using **Prim's** algorithm.



```
1 #include <stdio.h>
2 #define INF 999
3
4 int main() {
5     int n, i, j, min, u, v, total = 0;
6     printf("Enter number of vertices: ");
7     scanf("%d", &n);
8     int cost[n][n], visited[n];
9     printf("Enter cost matrix:\n");
10    for(i=0; i<n; i++)
11        for(j=0; j<n; j++)
12            scanf("%d", &cost[i][j]);
13    for(i=0; i<n; i++)
14        visited[i]=0;
15    visited[0]=1;
16    printf("\nEdges in MST:\n");
17    for(int e=0; e<n-1; e++) {
18        min=INF;
19        for(i=0; i<n; i++) {
20            if(visited[i]) {
21                for(j=0; j<n; j++) {
22                    if(!visited[j] && cost[i][j]<min) {
23                        min=cost[i][j];
24                        u=i;
25                        v=j;
26                    }
27                }
28            }
29            printf("%d -> %d : %d\n", u, v, min);
30            total+=min;
31            visited[v]=1;
32        }
33        printf("Minimum Cost = %d\n", total);
34    }
```

Logs and others

CodeBlocks x Search results x C/C++ x Build log x Build messages x CppCheck/Ver++ x CppCheck/Ver++ messages x Cscope x Debugger x DoryBlocks x Fortran info x Closed files list x Thread search x

File Line Message

Build file: "no target" in "no project" (compiler: unknown) ---

Build finished: 0 error(s), 0 warning(s) (0 minute(s), 1 second(s)) ---

D:\Sem 4\ADA\LAB\Prim's algorithm.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 35, Col 2, Pos 931 Insert Read/Write default

```
1 #include <stdio.h>
2 #define INF 999
3
4 int main() {
5     int n, i, j, min, u, v, total;
6     printf("Enter number of vertices: ");
7     scanf("%d", &n);
8     int cost[n][n], visited[n];
9     printf("Enter cost matrix:\n");
10    for(i=0; i<n; i++)
11        for(j=0; j<n; j++)
12            scanf("%d", &cost[i][j]);
13    for(i=0; i<n; i++)
14        visited[i]=0;
15    visited[0]=1;
16    printf("\nEdges in MST:\n");
17    for(int e=0; e<n-1; e++) {
18        min=INF;
19        for(i=0; i<n; i++) {
20            if(visited[i]) {
21                for(j=0; j<n; j++) {
22                    if(!visited[j] && cost[i][j]<min) {
23                        min=cost[i][j];
24                        u=i;
25                        v=j;
26                    }
27                }
28            }
29            printf("%d -> %d : %d\n", u, v, min);
30            total+=min;
31            visited[v]=1;
32        }
33        printf("Minimum Cost = %d\n", total);
34    }
```

Logs and others

CodeBlocks x Search results x C/C++ x Build log x Build messages x CppCheck/Ver++ x CppCheck/Ver++ messages x Cscope x Debugger x DoryBlocks x Fortran info x Closed files list x Thread search x

File Line Message

Build file: "no target" in "no project" (compiler: unknown) ---

Build finished: 0 error(s), 0 warning(s) (0 minute(s), 0 second(s)) ---

D:\Sem 4\ADA\LAB\Prim's algorithm.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 35, Col 2, Pos 931 Insert Read/Write default

Find Minimum Cost Spanning Tree of a given undirected graph using **Kruskal's** algorithm.

The screenshot shows the Code::Blocks IDE with the file "Kruskal's algorithm.c" open. The code implements Kruskal's algorithm to find the Minimum Cost Spanning Tree (MST) of an undirected graph. The code includes a find function for disjoint sets, a union function, and a main function that reads the number of vertices, the cost matrix, and prints the edges in the MST and the minimum cost.

```
1 #include <stdio.h>
2 #define INF 999
3 int parent[20];
4 int find(int i) {
5     while(parent[i])
6         i=parent[i];
7     return i;
8 }
9 int uni(int i, int j) {
10    if(i!=j) {
11        parent[j]=i;
12        return 1;
13    }
14    return 0;
15 }
16 int main() {
17     int n,i,j,min,a,b,u,v,total=0,edges=0;
18     printf("Enter number of vertices: ");
19     scanf("%d",&n);
20     int cost[20][20];
21     printf("Enter cost matrix:\n");
22     for(i=0;i<n;i++)
23         for(j=0;j<n;j++)
24             scanf("%d",&cost[i][j]);
25     printf("\nEdges in MST:\n");
26     while(edges<n-1) {
27         min=INF;
28         for(i=0;i<n;i++) {
29             for(j=0;j<n;j++) {
30                 if(cost[i][j]<min) {
31                     min=cost[i][j];
32                     a=i;
33                     b=j;
```

The screenshot shows the Code::Blocks IDE with the same file "Kruskal's algorithm.c" open. A terminal window titled "D:\Sem 4\ADAN\LAB\Kruskal's algorithm.exe" is overlaid on the code editor, showing the program's execution. The user enters 5 for the number of vertices and a 5x5 cost matrix. The program outputs the edges in the MST and the minimum cost.

```
Enter number of vertices: 5
Enter cost matrix:
999 2 999 6 999
2 999 3 8 5
999 3 999 999 7
6 8 999 999 9
999 5 7 9 999
Edges in MST:
0 -> 1 : 2
1 -> 2 : 3
1 -> 4 : 5
0 -> 3 : 6
Minimum Cost = 16
Process returned 0 (0x0)   execution time : 48.440 s
Press any key to continue.
```

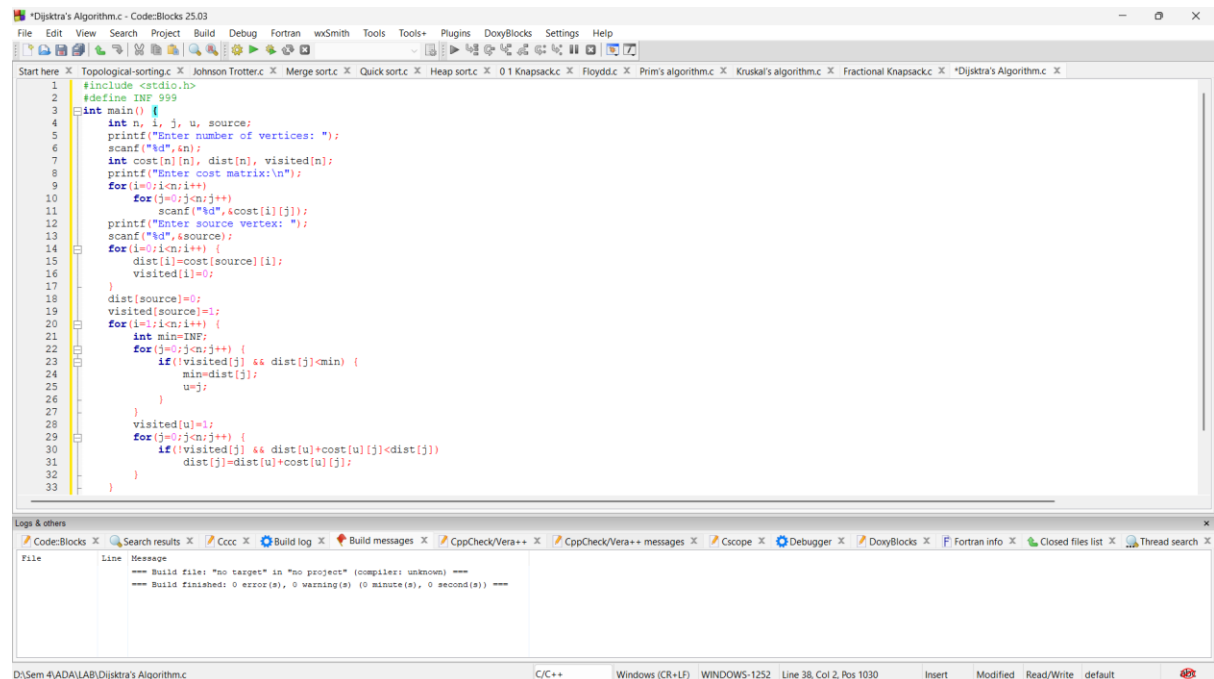
Implement Fractional Knapsack using Greedy technique.

```
1 #include <stdio.h>
2 struct Item {
3     int weight, profit;
4     float ratio;
5 };
6 int main() {
7     int n, i, j, capacity;
8     float maxProfit = 0.0;
9     printf("Enter number of items: ");
10    scanf("%d", &n);
11    struct Item item[n], temp;
12    printf("Enter weights and profits:\n");
13    for(i=0; i<n; i++) {
14        scanf("%d%d", &item[i].weight, &item[i].profit);
15        item[i].ratio = (float) item[i].profit / item[i].weight;
16    }
17    printf("Enter knapsack capacity: ");
18    scanf("%d", &capacity);
19    for(i=0; i<n-1; i++) {
20        for(j=0; j<n-i-1; j++) {
21            if(item[j].ratio < item[j+1].ratio) {
22                temp = item[j];
23                item[j] = item[j+1];
24                item[j+1] = temp;
25            }
26        }
27    }
28    for(i=0; i<n; i++) {
29        if(capacity >= item[i].weight) {
30            capacity -= item[i].weight;
31            maxProfit += item[i].profit;
32        } else {
33            maxProfit += item[i].ratio * capacity;
```

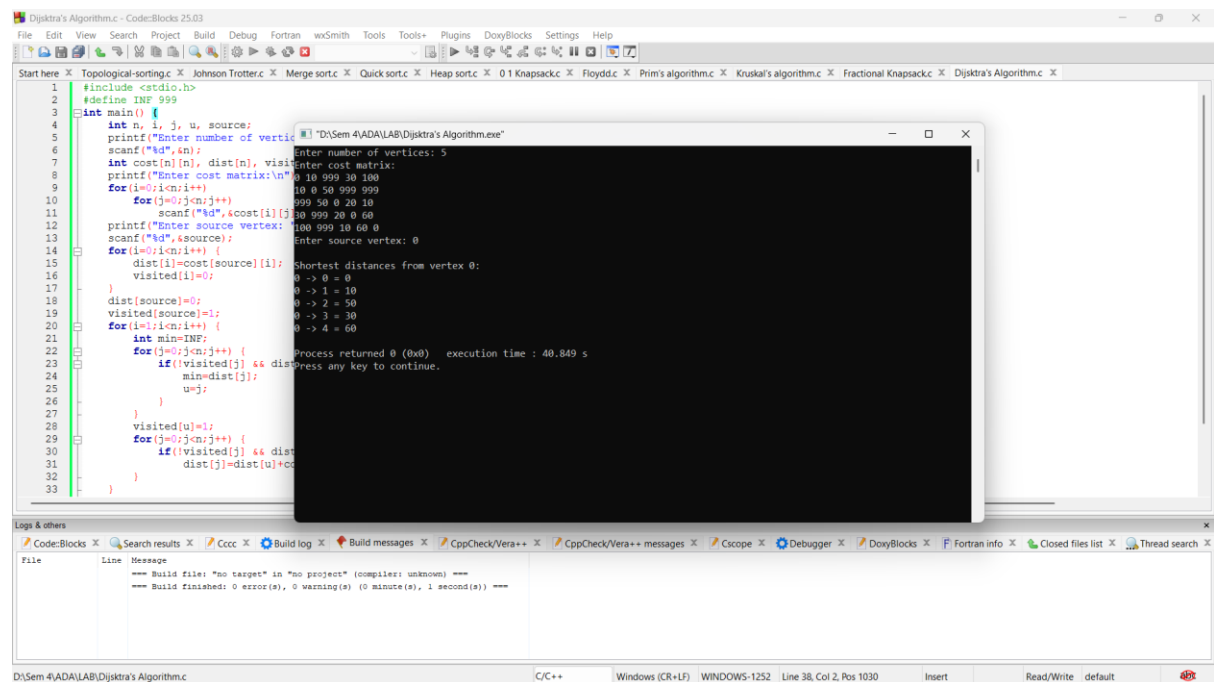
Execution Output:

```
Enter number of items: 3
Enter weights and profits:
10 60
20 100
30 120
Enter knapsack capacity: 50
Maximum Profit = 240.00
process returned 0 (0x0)  execution time : 27.170 s
Press any key to continue.
```

From a given vertex in a weighted connected graph, find shortest paths to other vertices using **Dijkstra's algorithm**.



```
1 #include <stdio.h>
2 #define INF 999
3 int main() {
4     int n, i, j, u, source;
5     printf("Enter number of vertices: ");
6     scanf("%d", &n);
7     int cost[n][n], dist[n], visited[n];
8     printf("Enter cost matrix:\n");
9     for(i=0; i<n; i++)
10         for(j=0; j<n; j++)
11             scanf("%d", &cost[i][j]);
12     printf("Enter source vertex: ");
13     scanf("%d", &source);
14     for(i=0; i<n; i++) {
15         dist[i]=cost[source][i];
16         visited[i]=0;
17     }
18     dist[source]=0;
19     visited[source]=1;
20     for(i=1; i<n; i++) {
21         int min=INF;
22         for(j=0; j<n; j++) {
23             if(!visited[j] && dist[j]<min) {
24                 min=dist[j];
25                 u=j;
26             }
27         }
28         visited[u]=1;
29         for(j=0; j<n; j++) {
30             if(!visited[j] && dist[u]+cost[u][j]<dist[j])
31                 dist[j]=dist[u]+cost[u][j];
32         }
33     }
34 }
```



```
Enter number of vertices: 5
Enter cost matrix:
10 999 30 100
10 0 50 999 999
999 50 0 20 10
30 999 20 0 60
100 999 10 60 0
Enter source vertex: 0
Shortest distances from vertex 0:
0 -> 0 = 0
0 -> 1 = 10
0 -> 2 = 50
0 -> 3 = 30
0 -> 4 = 60
Process returned 0 (0x0)   execution time : 40.849 s
Press any key to continue.
```

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>
#include <stdlib.h>
int x[20], n;

int place(int k, int i) {
    int j;
    for(j=1; j<k; j++)
        if(x[j]==i || abs(x[j]-i)==abs(j-k))
            return 0;
    return 1;
}

void printSolution() {
    int i, j;
    printf("\nSolution:\n");
    for(i=1; i<=n; i++) {
        for(j=1; j<=n; j++) {
            if(x[i]==j)
                printf("Q ");
            else
                printf(" ");
        }
        printf("\n");
    }
}

void nQueens(int k) {
    int i;
    for(i=1; i<=n; i++) {
        if(place(k, i)) {
            x[k]=i;
            if(k==n)
                printSolution();
            else
                nQueens(k+1);
        }
    }
}
```

Logs & others

Code::Blocks Search results C/C++ Build log Build messages CppCheck/Ver++ CppCheck/Ver++ messages Cscope Debugger DoryBlocks Fortran info Closed files list Thread search

File Line Message

==== Build file: "no target" in "no project" (compiler: unknown) ====

==== Build finished: 0 error(s), 0 warning(s) (0 minute(s), 1 second(s)) ====

D:\Sem 4\ADA\LAB\N Queens.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 2, Col 20, Pos 39 Insert Read/Write default

N Queens.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoryBlocks Settings Help

N Queens.c

```
#include <stdio.h>
#include <stdlib.h>
int x[20], n;

int place(int k, int i) {
    int j;
    for(j=1; j<k; j++)
        if(x[j]==i || abs(x[j]-i)==abs(j-k))
            return 0;
    return 1;
}

void printSolution() {
    int i, j;
    printf("\nSolution:\n");
    for(i=1; i<=n; i++) {
        for(j=1; j<=n; j++) {
            if(x[i]==j)
                printf("Q ");
            else
                printf(" ");
        }
        printf("\n");
    }
}

void nQueens(int k) {
    int i;
    for(i=1; i<=n; i++) {
        if(place(k, i)) {
            x[k]=i;
            if(k==n)
                printSolution();
            else
                nQueens(k+1);
        }
    }
}
```

D:\Sem 4\ADA\LAB\N Queens.exe

Enter value of N: 4

Solution:

```
Q . . .
. Q . .
. . Q .
. . . Q
```

Solution:

```
. Q . .
. . Q .
Q . . .
. . . Q
```

Process returned 0 (0x0) execution time : 16.002 s

Press any key to continue.

Logs & others

Code::Blocks Search results C/C++ Build log Build messages CppCheck/Ver++ CppCheck/Ver++ messages Cscope Debugger DoryBlocks Fortran info Closed files list Thread search

File Line Message

==== Build file: "no target" in "no project" (compiler: unknown) ====

==== Build finished: 0 error(s), 0 warning(s) (0 minute(s), 1 second(s)) ====

D:\Sem 4\ADA\LAB\N Queens.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 2, Col 20, Pos 39 Insert Read/Write default

Leetcode:

Github link: <https://github.com/rohans-cs24/ADA>

Date: 25-03-2026

75. Sort Colors

Given an array `nums` with `n` objects colored red, white, or blue, sort them **in-place** so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers `0`, `1`, and `2` to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

Input: `nums = [2,0,2,1,1,0]`
Output: `[0,0,1,1,2,2]`

Example 2:

Input: `nums = [2,0,1]`
Output: `[0,1,2]`

Constraints:

- `n == nums.length`
- `1 <= n <= 300`
- `nums[i]` is either `0`, `1`, or `2`.

Follow up: Could you come up with a one-pass algorithm using only constant extra space?

See this question in a real interview before? 1/5

21.5K 659 0 Online

```
1 #include <stdio.h>
2
3 void sortColors(int* nums, int numsSize) {
4     int low = 0, mid = 0, high = numsSize - 1;
5
6     while (mid <= high) {
7         if (nums[mid] == 0) {
8             int temp = nums[low];
9             nums[low] = nums[mid];
10            nums[mid] = temp;
11            low++;
12            mid++;
13        } else if (nums[mid] == 1) {
14            mid++;
15        } else {
16            int temp = nums[mid];
17            nums[mid] = nums[high];
18            nums[high] = temp;
19            high--;
20        }
21    }
22 }
```

Accepted Runtime: 0 ms

Case 1 Case 2

Input: `nums = [2,0,2,1,1,0]`

Output: `[0,0,1,1,2,2]`

Expected: `[0,0,1,1,2,2]`

Contribute a testcase

88. Merge Sorted Array

You are given two integer arrays `nums1` and `nums2`, sorted in **non-decreasing order**, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in **non-decreasing order**.

The final sorted array should not be returned by the function, but instead be stored **inside the array** `nums1`. To accommodate this, `nums1` has a length of `m + n`, where the first `m` elements denote the elements that should be merged, and the last `n` elements are set to `0` and should be ignored. `nums2` has a length of `n`.

Example 1:

Input: `nums1 = [1,2,3,0,0,0]`, `m = 3`, `nums2 = [2,5,6]`, `n = 3`
Output: `[1,2,2,3,5,6]`
Explanation: The arrays we are merging are `[1,2,3]` and `[2,5,6]`.
The result of the merge is `[1,2,2,3,5,6]` with the underlined elements coming from `nums1`.

Example 2:

Input: `nums1 = [1]`, `m = 1`, `nums2 = []`, `n = 0`
Output: `[1]`
Explanation: The arrays we are merging are `[1]` and `[]`.
The result of the merge is `[1]`.

Example 3:

Input: `nums1 = [0]`, `m = 0`, `nums2 = [1]`, `n = 1`
Output: `[1]`
Explanation: The arrays we are merging are `[]` and `[1]`.
The result of the merge is `[1]`.
Note that because `m = 0`, there are no elements in `nums1`. The `0` is only there to ensure the merge result can fit in `nums1`.

Constraints:

- `nums1.length == m + n`
- `0 <= m <= 100`
- `0 <= n <= 100`
- `1 <= m + n <= 200`
- `nums1[i]` and `nums2[i]` are in the range `[0, 100]`

18.9K 943 0 Online

```
1 void merge(int* nums1, int nums1Size, int m, int* nums2, int nums2Size,
2           int n) {
3     int i = m - 1;
4     int j = n - 1;
5     int k = m + n - 1;
6
7     while (i >= 0 && j >= 0) {
8         if (nums1[i] > nums2[j]) {
9             nums1[k] = nums1[i];
10            i--;
11        } else {
12            nums1[k] = nums2[j];
13            j--;
14        }
15        k--;
16    }
17    while (i <= 0 && j >= 0) {
18        nums1[k] = nums2[j];
19        j--;
20        k--;
21    }
22 }
```

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input: `nums1 = [1,2,3,0,0,0]`

`m = 3`

`nums2 = [2,5,6]`

`n = 3`

Output: `[1,2,2,3,5,6]`

Problem List

215. Kth Largest Element in an Array

Solved

Medium

Topics

Companies

Given an integer array `nums` and an integer `k`, return the k^{th} largest element in the array.

Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element.

Can you solve it without sorting?

Example 1:

Input: `nums = [3,2,1,5,6,4]`, `k = 2`
Output: `5`

Example 2:

Input: `nums = [3,2,3,1,2,4,5,6]`, `k = 4`
Output: `4`

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Samsung

Galaxy Buds4 Pro

Seen this question in a real interview before?

1/5

Accepted

3,547,541 / 5.2M

Acceptance Rate 68.8%

Topics

18.7K

381

0 Online

Code

Accepted

```
17 if (right < n && heap[right] < heap[smallest]) smallest = right;
18
19 if (smallest != i) {
20     swap(heap[i], heap[smallest]);
21     heapify(heap, n, smallest);
22 }
23
24 int findKthLargest(int* nums, int numsSize, int k) {
25     int* heap = (int*)malloc(sizeof(int) * k);
26     int heapSize = 0;
27
28     for (int i = 0; i < numsSize; i++) {
29         if (heapSize < k) {
30             heap[heapSize++] = nums[i];
31             if (heapSize == k) {
32                 for (int j = k/2 - 1; j >= 0; j--) {
33                     swap(heap[j], heap[j+1]);
34                     heapify(heap, k, j);
35                 }
36             }
37         }
38     }
39     return heap[0];
40 }
```

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

`nums = [3,2,1,5,6,4]`
`k = 2`

Output

`5`

Expected

`5`

rohan_s_m_14

Access all features with our Premium subscription

My Lists

Notebook

Progress

Points

Try New Features

Orders

My Playgrounds

Settings

Appearance

Sign Out

Problem List

268. Missing Number

Solved

Easy

Topics

Companies

Given an array `nums` containing n distinct numbers in the range $[0, n]$, return the only number in the range that is missing from the array.

Example 1:

Input: `nums = [3,0,1]`
Output: `2`
Explanation:
 $n = 3$ since there are 3 numbers, so all numbers are in the range $[0, 3]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 2:

Input: `nums = [0,1]`
Output: `2`
Explanation:
 $n = 2$ since there are 2 numbers, so all numbers are in the range $[0, 2]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 3:

Input: `nums = [9,6,4,2,3,5,7,0,1]`
Output: `8`
Explanation:
 $n = 9$ since there are 9 numbers, so all numbers are in the range $[0, 9]$. 8 is the missing number in the range since it does not appear in `nums`.

14.3K

402

0 Online

Code

Accepted

```
1 int missingNumber(int* nums, int numsSize) {
2     int n = numsSize;
3     int expected = (n * (n+1))/2;
4     int actual = 0;
5     for (int i = 0; i < numsSize; i++) {
6         actual += nums[i];
7     }
8     return expected - actual;
9 }
```

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

`nums = [3,0,1]`

Output

`2`

Expected

`2`

rohan_s_m_14

Access all features with our Premium subscription

My Lists

Notebook

Progress

Points

Try New Features

Orders

My Playgrounds

Settings

Appearance

Sign Out

Date: 08-05-2026

879. Profitable Schemes

There is a group of n members, and a list of various crimes they could commit. The i^{th} crime generates a `profit[i]` and requires `group[i]` members to participate in it. If a member participates in one crime, that member can't participate in another crime.

Let's call a **profitable scheme** any subset of these crimes that generates at least `minProfit` profit, and the total number of members participating in that subset of crimes is at most `n`.

Return the number of schemes that can be chosen. Since the answer may be very large, return it modulo $10^9 + 7$.

Example 1:
Input: `n = 5, minProfit = 3, group = [2,2], profit = [2,3]`
Output: 2
Explanation: To make a profit of at least 3, the group could either commit crimes 0 and 1, or just crime 1. In total, there are 2 schemes.

Example 2:
Input: `n = 10, minProfit = 5, group = [2,3,5], profit = [6,7,8]`
Output: 7
Explanation: To make a profit of at least 5, the group could commit any crimes, as long as they commit one. There are 7 possible schemes: (0), (1), (2), (0,1), (0,2), (1,2), and (0,1,2).

Constraints:

- $1 \leq n \leq 100$
- $0 \leq \text{minProfit} \leq 100$
- $1 \leq \text{group.length} \leq 100$
- $1 \leq \text{group}[i] \leq 100$
- $\text{profit.length} = \text{group.length}$
- $0 \leq \text{profit}[i] \leq 100$

```
1 #define MOD 1000000007
2 int profitableSchemes(int n, int minProfit, int* group, int groupSize,
3                       int* profit, int profitSize) {
4     int dp[105][105] = {0};
5     dp[0][0] = 1;
6     for (int k = 0; k < groupSize; k++) {
7         int g = group[k];
8         int p = profit[k];
9         for (int members = n; members >= g; members--) {
10             for (int prof = minProfit; prof >= 0; prof--) {
11                 int newProfit = prof + p;
12                 if (newProfit > minProfit) newProfit = minProfit;
13                 dp[members][newProfit] = (dp[members][newProfit] + dp[members-g][prof]) % MOD;
14             }
15         }
16     }
17     int ans = 0;
18     for (int members = 0; members <= n; members++) {
19         for (int prof = minProfit; prof <= 100; prof++) {
20             ans = (ans + dp[members][prof]) % MOD;
21         }
22     }
23     return ans;
24 }
```

Date: 29-05-2026

1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance

There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`.

Return the city with the smallest number of cities that are reachable through some path and whose distance is at most `distanceThreshold`. If there are multiple such cities, return the city with the greatest number.

Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

Example 1:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define INF 1000000000
5
6 int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize, int distanceThreshold) {
7     int dist[105][105];
8     // Initialize distance matrix
9     for (int i = 0; i < n; i++) {
10         for (int j = 0; j < n; j++) {
11             if (i == j) dist[i][j] = 0;
12             else dist[i][j] = INF;
13         }
14     }
15     // Fill the distance matrix with the given edges
16     for (int i = 0; i < edgesSize; i++) {
17         int u = edges[i][0], v = edges[i][1], w = edges[i][2];
18         dist[u][v] = w;
19         dist[v][u] = w;
20     }
21     // Floyd-Warshall algorithm to find shortest paths between all pairs of cities
22     for (int k = 0; k < n; k++) {
23         for (int i = 0; i < n; i++) {
24             for (int j = 0; j < n; j++) {
25                 if (dist[i][k] + dist[k][j] < dist[i][j]) {
26                     dist[i][j] = dist[i][k] + dist[k][j];
27                 }
28             }
29         }
30     }
31     int minNeighbors = n;
32     int bestCity = -1;
33     for (int i = 0; i < n; i++) {
34         int count = 0;
35         for (int j = 0; j < n; j++) {
36             if (i != j && dist[i][j] <= distanceThreshold) {
37                 count++;
38             }
39         }
40         if (count < minNeighbors) {
41             minNeighbors = count;
42             bestCity = i;
43         }
44     }
45     return bestCity;
46 }
```